SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4307

COURSE OUTCOME COVERED

CO -2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Design Task

Weightage: 20%

QUESTIONS: **Total Marks: 20**

Question 1: Responsive Navigation Bar Design - Marks(4)

Suppose that an e-commerce website requires a responsive navigation bar that adapts across desktop, tablet, and mobile devices.
(a) Identify key components required in a navigation bar for an e-commerce website.
(b) Design a responsive navigation bar using appropriate CSS techniques.
(c) Demonstrate how the navigation adapts to smaller screens (e.g., hamburger menu).
(d) Evaluate the usability and suggest improvements for better user experience.

Question 2: Product Hover Effect - Marks(4)

Suppose that an e-commerce website wants to enhance user interaction through visual effects on product items.
(a) Identify suitable CSS properties for hover effects.
(b) Suggest appropriate **CSS animation or transition** for product interaction.
(c) Design a hover effect for a product card.
(d) Evaluate performance and user experience considerations.

Question 3: Layout Selection (Flexbox vs Grid) - Marks(4)

Suppose that a dashboard layout needs to be designed for a web application.
(a) Identify layout requirements of a dashboard (rows, columns, alignment).
(b) Compare **Flexbox and Grid** based on layout needs.
(c) Choose the most suitable layout technique and justify your choice.
(d) Evaluate the scalability and responsiveness of the chosen approach.

Question 4: Mobile-First Blog Layout - Marks(4)

Suppose that a blog page must be designed following a mobile-first approach.
(a) Identify key sections of a blog layout.
(b) Design a mobile-first layout using CSS.
(c) Demonstrate how the layout scales for larger screens using media queries.
(d) Evaluate accessibility and readability of the design.

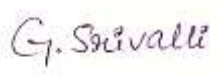
Question 5: JavaScript Event Handling for Dynamic Menu - Marks(4)

Suppose that a website includes a dynamic menu that responds to user interactions.
(a) Identify appropriate JavaScript events for menu interaction.
(b) Design event handling logic for menu toggle functionality.
(c) Implement interaction for user actions (click/hover).
(d) Evaluate performance and suggest improvements.

Code of Conduct:

I G. Srivalli (192365014) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Concept	25	Demonstrates complete understanding of design requirements and concepts	Good understanding with minor gaps	Basic understanding with some errors	Poor understanding or incorrect concepts
Design / Implementation	30	Well-structured, efficient, and responsive design implementation	Correct implementation with minor issues	Basic implementation with inconsistencies	Incorrect or incomplete implementation
Use of Tools (CSS/JS)	20	Effective and advanced use of tools/techniques	Appropriate use of tools	Limited or inconsistent use	Incorrect or minimal use
Analysis & Evaluation	15	Insightful analysis with strong justification and optimization	Good analysis with justification	Basic analysis with limited depth	Poor or no analysis
Documentation / Clarity	10	Clear, well-structured, and properly presented solution	Mostly clear with minor issues	Basic clarity, missing details	Poor or unclear documentation

Question 1: Responsive Navigation Bar Design

(a) Key Components of an E-Commerce Navigation Bar

- **Brand Identity / Logo:** Anchors the site and provides a one-click return path to the homepage.
- **Category Navigation Links:** Primary links to departmental hierarchies (e.g., *Men, Women, Electronics, Deals*).
- **Global Search Bar:** Direct input field enabling immediate product discovery, auto-suggestions, and category filters.
- **User Utility Actions:** Account/Login status, Wishlist counter, and an interactive Shopping Cart badge displaying active item counts.
- **Mobile Toggle (Hamburger Button):** An accessible control element hidden on large screens that expands navigation drawers on constrained viewports.

(b) Responsive Navigation Bar Implementation (HTML & CSS)

HTML

```
<header class="navbar-wrapper">
  <div class="logo"><a href="#">ShopSphere</a></div>
  <input type="checkbox" id="nav-toggle" class="nav-toggle-input">
  <label for="nav-toggle" class="hamburger-icon" aria-label="Toggle navigation">
    <span></span>
    <span></span>
    <span></span>
  </label>
  <nav class="nav-menu">
    <ul class="nav-links">
      <li><a href="#new">New Arrivals</a></li>
      <li><a href="#men">Men</a></li>
      <li><a href="#women">Women</a></li>
      <li><a href="#sale">Sale</a></li>
    </ul>
    <div class="nav-utilities">
      <input type="search" placeholder="Search products..." class="search-box">
      <a href="#cart" class="cart-btn">Cart (0)</a>
```

```
    </div>
  </nav>
</header>
```

CSS

```
* { box-sizing: border-box; margin: 0; padding: 0; }
.navbar-wrapper {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
  background-color: #ffffff;
  border-bottom: 1px solid #e2e8f0;
  position: sticky;
  top: 0;
  z-index: 1000;
}
.logo a { font-size: 1.5rem; font-weight: 700; color: #1e293b; text-decoration: none; }
.nav-menu { display: flex; align-items: center; gap: 2rem; }
.nav-links { display: flex; list-style: none; gap: 1.5rem; }
.nav-links a { text-decoration: none; color: #475569; font-weight: 500; transition: color
0.2s ease; }
.nav-links a:hover { color: #2563eb; }
.nav-utilities { display: flex; align-items: center; gap: 1rem; }
.search-box { padding: 0.5rem 0.8rem; border: 1px solid #cbd5e1; border-radius:
6px; }
.cart-btn { text-decoration: none; font-weight: 600; color: #2563eb; }
.nav-toggle-input, .hamburger-icon { display: none; }
```

(c) Adaptation to Smaller Screens (Mobile Hamburger Transition)

Using CSS Media Queries (@media (max-width: 768px)), desktop flex elements collapse into an off-screen drawer triggered via the accessible checkbox hack:

CSS

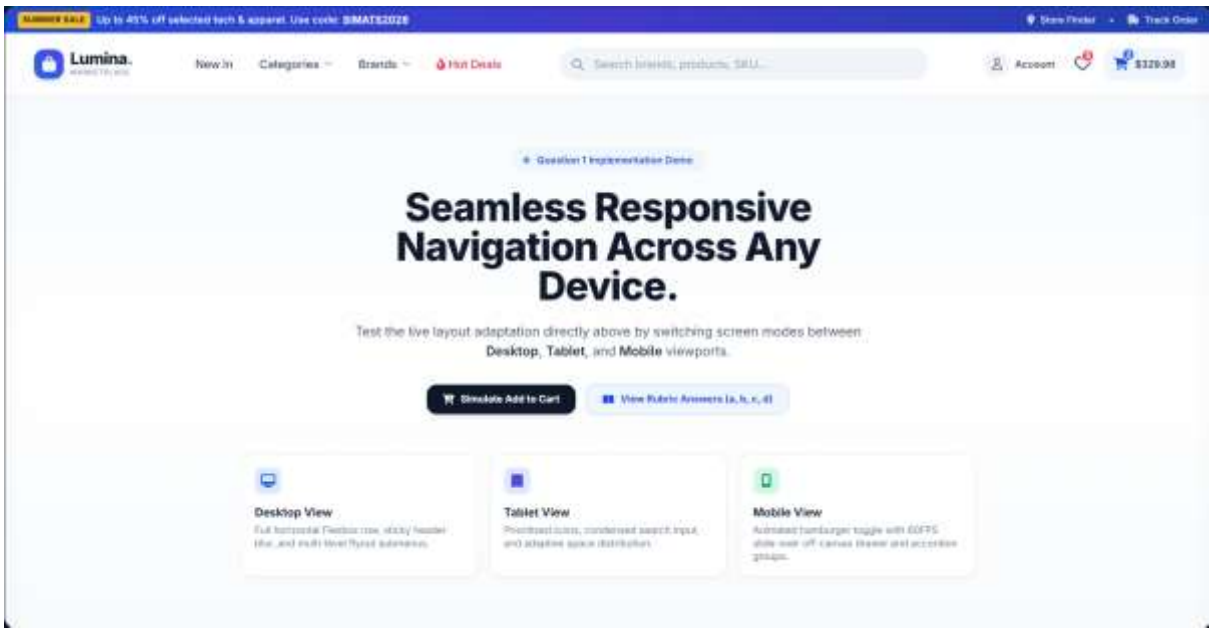
```
@media (max-width: 768px) {
  .hamburger-icon {
```

```
  display: flex;
  flex-direction: column;
  justify-content: space-between;
  width: 26px;
  height: 20px;
  cursor: pointer;
}
.hamburger-icon span {
  height: 3px;
  width: 100%;
  background-color: #1e293b;
  border-radius: 2px;
}
.nav-menu {
  position: absolute;
  top: 100%;
  left: 0;
  right: 0;
  flex-direction: column;
  align-items: flex-start;
  background: #ffffff;
  padding: 1.5rem;
  gap: 1.25rem;
  box-shadow: 0 10px 15px -3px rgba(0, 0, 0, 0.1);
  transform: translateY(-150%);
  opacity: 0;
  pointer-events: none;
  transition: transform 0.3s ease, opacity 0.3s ease;
}
.nav-links { flex-direction: column; width: 100%; gap: 1rem; }
.nav-utilities { flex-direction: column; width: 100%; align-items: stretch; }
.search-box { width: 100%; }
/* Trigger menu visibility when checkbox is checked */
.nav-toggle-input:checked ~ .nav-menu {
```

```
    transform: translateY(0);
    opacity: 1;
    pointer-events: auto;
  }
}
```

(d) Usability Evaluation and Improvements

- **Touch Target Sizing:** Ensure all touchable links and buttons maintain minimum tap dimensions of $48\text{px} \times 48\text{px}$ to prevent mis-clicks.
- **Sticky Positioning & Visual Clutter:** Retain sticky navigation on mobile, but auto-collapse the global search into an expandable overlay icon to preserve vertical screen estate.
- **Accessibility (a11y):** In production setups, replace pure CSS checkbox toggles with semantic <button> elements driven by aria-expanded and aria-controls to properly announce status to screen readers.



Question 2: Product Hover Effect

(a) Suitable CSS Properties for Hover Effects

- transform (scale, translateY): Modifies scale and vertical offset without triggering page layout recalibrations.
- box-shadow: Adds visual elevation and depth to indicate interactivity.
- opacity: Controls visual transitions for overlays, such as quick-view buttons or secondary product imagery.
- filter (brightness(), contrast()): Enhances visual feedback on product images.

(b) Appropriate Animation and Transition Choice

- **Properties to Transition:** transform, box-shadow, and opacity. These are GPU-accelerated and avoid triggering costly browser Reflow (Layout) and Repaint cycles.
- **Timing Function & Duration:** Use cubic-bezier(0.4, 0, 0.2, 1) (standard ease-in-out) with a duration between 200ms and 300ms. Durations above 400ms feel sluggish, while values below 100ms appear abrupt.

(c) Implementation of Product Card Hover Effect

HTML

```
<div class="product-card">
  <div class="card-media">
    <img src="sneaker.jpg" alt="Running Shoes" class="product-img">
    <button class="quick-view-btn">Quick Add</button>
  </div>
  <div class="card-details">
    <span class="category">Footwear</span>
    <h3 class="title">Aero Pro Runners</h3>
    <p class="price">$129.99</p>
  </div>
</div>
```

CSS

```
.product-card {
  width: 280px;
```

```
  background: #ffffff;
  border-radius: 12px;
  overflow: hidden;
  border: 1px solid #e2e8f0;
  box-shadow: 0 1px 3px rgba(0, 0, 0, 0.05);
  transition: transform 0.25s cubic-bezier(0.4, 0, 0.2, 1),
              box-shadow 0.25s cubic-bezier(0.4, 0, 0.2, 1);
  will-change: transform, box-shadow;
}
.card-media {
  position: relative;
  overflow: hidden;
}
.product-img {
  width: 100%;
  height: 240px;
  object-fit: cover;
  transition: transform 0.3s ease;
}
.quick-view-btn {
  position: absolute;
  bottom: 12px;
  left: 50%;
  transform: translate(-50%, 20px);
  opacity: 0;
  background: #2563eb;
  color: white;
  border: none;
  padding: 0.6rem 1.2rem;
  border-radius: 6px;
  cursor: pointer;
  transition: transform 0.25s ease, opacity 0.25s ease;
}
.card-details { padding: 1rem; }
```

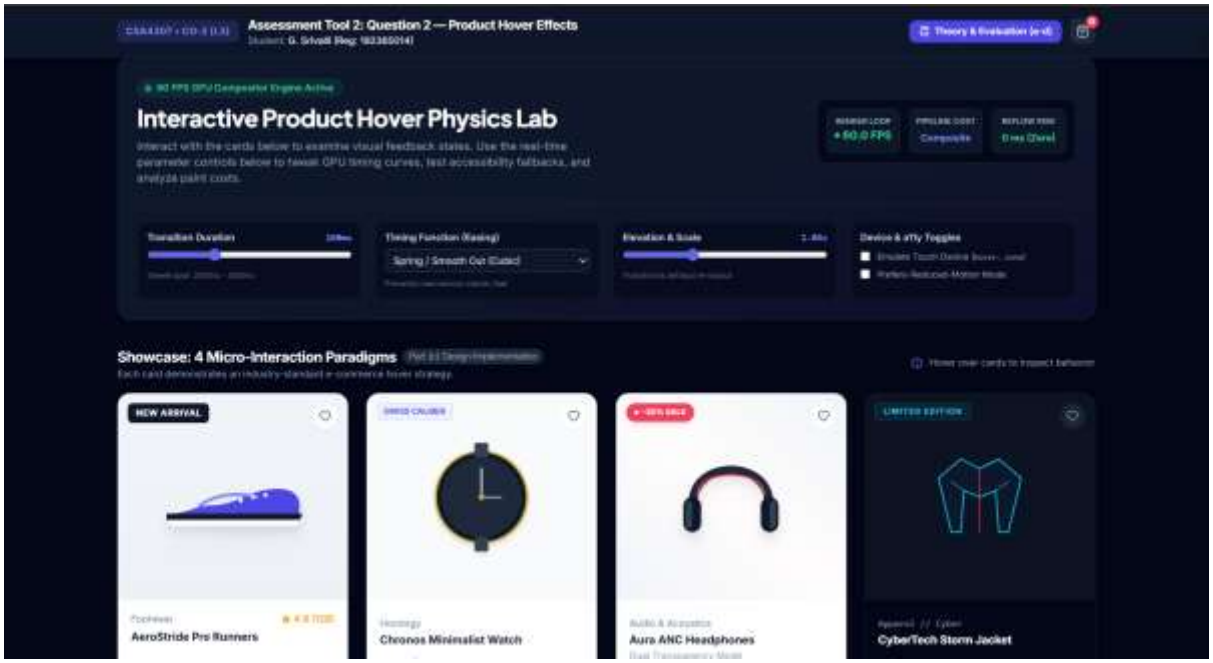
```
/* Interactive Hover State */
.product-card:hover {
  transform: translateY(-6px);
  box-shadow: 0 12px 20px -4px rgba(0, 0, 0, 0.12);
}
.product-card:hover .product-img {
  transform: scale(1.04);
}
.product-card:hover .quick-view-btn {
  transform: translate(-50%, 0);
  opacity: 1;
}
```

(d) Performance and User Experience Considerations

- **Hardware Acceleration:** Restrict animations to transform and opacity to keep rendering on the GPU Compositor thread and ensure smooth 60 FPS playback. Avoid animating dimensions (width, height, top, margin).
- **Touch-Screen Parity:** Mobile devices lack a persistent hover state. Tapping can trigger sticky hover styles. Use @media (hover: hover) media queries to only apply these hover enhancements on pointer-enabled hardware, falling back to static buttons on mobile.
- **Reduced Motion:** Provide accessibility support for users sensitive to motion sickness:

CSS

```
@media (prefers-reduced-motion: reduce) {
  .product-card, .product-img, .quick-view-btn {
    transition: none !important;
    transform: none !important;
  }
}
```



Question 3: Layout Selection (Flexbox vs Grid)

(a) Dashboard Layout Requirements

- **Header / Top Bar:** Full-width fixed container housing breadcrumbs, notifications, and profile controls.
- **Collapsible Sidebar:** Fixed-width vertical track containing core application navigation.
- **Analytical Metrics Row:** Uniform metric tiles (KPI cards) distributed evenly across horizontal space.
- **Main Data Area:** Multi-column arrangements featuring uneven elements, such as a large chart panel (spanning 2 columns) placed beside a real-time event feed.

(b) Comparison: Flexbox vs. CSS Grid

Feature	CSS Flexbox	CSS Grid
Dimensionality	One-dimensional (processes either a row *or* a column at a time).	Two-dimensional (coordinates rows *and* columns simultaneously).

Approach	Content-driven: structural child widths dictate flow.	Layout-driven: predefined tracks dictate child placement.
Grid Lines & Spanning	Cannot align child items directly across separate, unrelated flex containers.	Native cross-axis alignment (grid-column: span 2, grid-row).
Ideal Role	Component-level alignment (toolbars, card headers, button clusters).	Page-level scaffolding and multi-card analytical layouts.

(c) Selection and Justification

Selected Technique: CSS Grid for macro layout scaffolding, coupled with **Flexbox** for internal component arrangements.

Justification: A dashboard requires coordinated row and column controls. CSS Grid enables distinct layout definition via grid-template-areas:

CSS

```
.dashboard-container {
  display: grid;
  grid-template-columns: 260px 1fr;
  grid-template-rows: 64px 1fr;
  grid-template-areas:
    "sidebar header"
    "sidebar content";
  min-height: 100vh;
}
.sidebar { grid-area: sidebar; }
.header { grid-area: header; }
.content {
  grid-area: content;
  display: grid;
  grid-template-columns: repeat(12, 1fr);
  gap: 1.5rem;
  padding: 1.5rem;
}
```

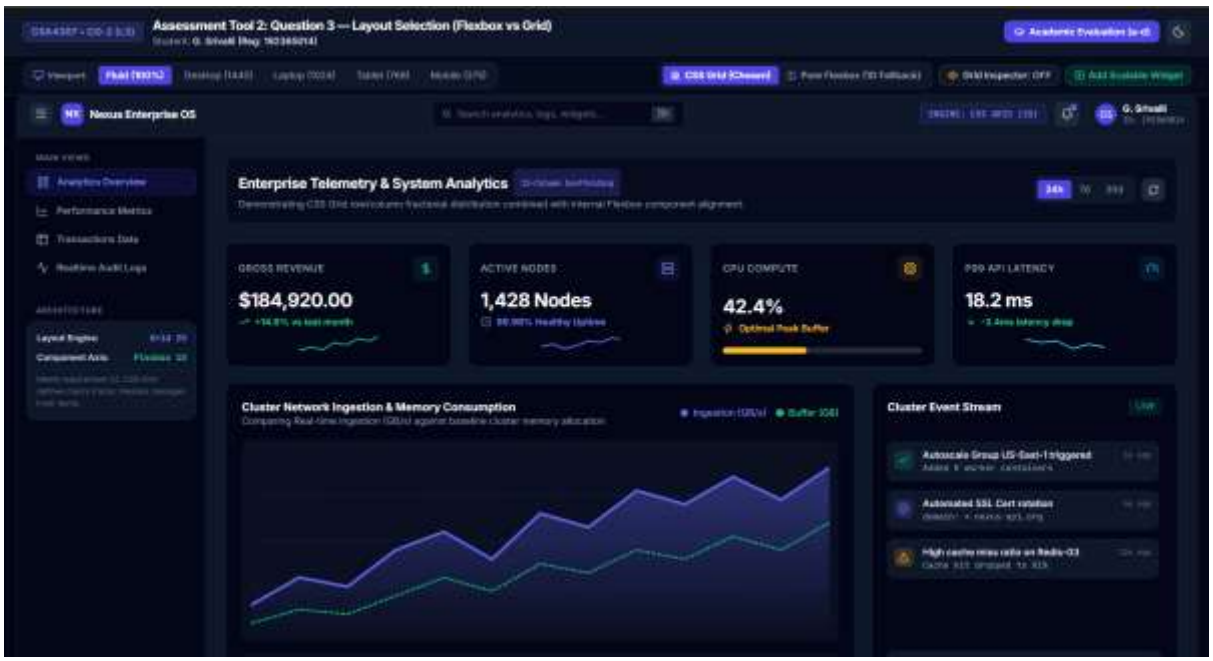
.metric-card { grid-column: span 3; }
.chart-panel { grid-column: span 8; }
.activity-feed { grid-column: span 4; }

(d) Scalability and Responsiveness Evaluation

- **Scalability:** The dashboard structure easily supports new widgets. Changing an analytical tile from a single cell to a wide graphic requires only updating grid-column: span X without rewriting structural HTML markup.
- **Responsive Reflow:** Mobile adaptations require only a simple media query reassignment:

CSS

```
@media (max-width: 900px) {
  .dashboard-container {
    grid-template-columns: 1fr;
    grid-template-areas: "header" "content";
  }
  .sidebar { display: none; } /* or convert to a drawer */
  .metric-card, .chart-panel, .activity-feed { grid-column: span 12; }
}
```



Question 4: Mobile-First Blog Layout

(a) Key Sections of a Blog Layout

- **Site Header:** Masthead branding, topic tags, and reading progress indicator.
- **Featured Article Header:** Publication date, estimated reading time, author metadata, and primary post title (H_1).
- **Main Editorial Content:** Structured long-form prose, blockquotes, inline imagery, and code snippets.
- **Contextual Sidebar:** Author biography, sticky table of contents (TOC), and newsletter subscription forms.
- **Footer:** Related article recommendations, copyright notices, and secondary navigational links.

(b) Mobile-First Layout Implementation (CSS)

In mobile-first design, base rules establish a single-column layout without media queries:

HTML

```
<div class="blog-wrapper">
  <header class="blog-header">
    <h1>The Architecture of Modern Web Interfaces</h1>
    <div class="meta">Published Oct 24 • 6 min read</div>
  </header>
  <main class="blog-main">
    <article class="post-content">
      <p>Responsive design has evolved beyond basic screen adjustments...</p>
    </article>
    <aside class="blog-sidebar">
      <div class="author-card">
        <h4>About the Author</h4>
        <p>Frontend engineer and web accessibility advocate.</p>
      </div>
    </aside>
  </main>
  <footer class="blog-footer">
```

```
    <p>&copy; 2026 TechBlog. All rights reserved.</p>
  </footer>
</div>
```

CSS

```
/* Mobile-first base styles (viewports < 768px) */
* { box-sizing: border-box; margin: 0; padding: 0; }
body {
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
  line-height: 1.7;
  color: #334155;
  background-color: #f8fafc;
}
.blog-wrapper {
  width: 100%;
  padding: 1.25rem;
  display: flex;
  flex-direction: column;
  gap: 1.5rem;
}
.blog-header h1 {
  font-size: 1.85rem;
  line-height: 1.25;
  color: #0f172a;
}
.meta { font-size: 0.875rem; color: #64748b; margin-top: 0.5rem; }
.blog-main {
  display: flex;
  flex-direction: column; /* Default single column for mobile */
  gap: 2rem;
}
.post-content { font-size: 1.05rem; }
.blog-sidebar {
  background: #ffffff;
  padding: 1.25rem;
```

```
  border-radius: 8px;
  border: 1px solid #e2e8f0;
}
```

(c) Desktop Scaling Using Media Queries (min-width**)**

Scale the layout upward using progressive min-width breakpoints:

CSS

```
/* Tablet & Desktop Reflow */
@media (min-width: 768px) {
  .blog-wrapper {
    max-width: 720px;
    margin: 0 auto;
    padding: 2.5rem 1rem;
  }
  .blog-header h1 { font-size: 2.5rem; }
}
@media (min-width: 1024px) {
  .blog-wrapper {
    max-width: 1140px;
  }
  .blog-main {
    display: grid;
    grid-template-columns: minmax(0, 1fr) 320px; /* Main content + sticky sidebar */
    gap: 3.5rem;
  }
  .blog-sidebar {
    position: sticky;
    top: 2rem;
    height: fit-content;
  }
}
```

(d) Accessibility and Readability Evaluation

- **Measure (Line Length):** By constraining desktop content tracks to a maximum width of ~ 70ch (characters per line), reading fatigue is minimized compared to unrestricted full-width text.
- **Typographic Hierarchy & Contrast:** High contrast between text (#0f172a / #334155) and background (#ffffff / #f8fafc) exceeds WCAG 2.1 AA requirements ($\geq 4.5:1$).
- **Fluid Scalability:** All font declarations and spacing use relative units (rem, ch), allowing layouts to scale cleanly with system-level browser zoom preferences.



Question 5: JavaScript Event Handling for Dynamic Menu

(a) Appropriate JavaScript Events for Menu Interaction

- 'click': Standard, cross-device event for toggles, drawers, and modal backdrops.
- 'keydown': Essential for accessibility; listens for Escape to close drawers or ArrowUp / ArrowDown / Tab to navigate submenus.
- 'pointerenter' / 'pointerleave': Reliable pointer-specific events for desktop flyout menus without introducing touch-delay anomalies.

(b) Event Handling Logic for Menu Toggle

HTML

```
<header>
  <button id="menu-btn" class="menu-trigger" aria-expanded="false" aria-
controls="mobile-nav">
    Menu
  </button>
  <nav id="mobile-nav" class="drawer-nav" hidden>
    <ul>
      <li><a href="#home">Home</a></li>
      <li><a href="#services">Services</a></li>
      <li><a href="#contact">Contact</a></li>
    </ul>
  </nav>
</header>
```

CSS

```
.drawer-nav { display: none; }
.drawer-nav.is-active { display: block; }
```

JavaScript

```
const menuBtn = document.getElementById('menu-btn');
const mobileNav = document.getElementById('mobile-nav');
function toggleNavigation(forceState) {
```

```
  const isExpanded = menuBtn.getAttribute('aria-expanded') === 'true';
  const shouldOpen = forceState !== undefined ? forceState : !isExpanded;
  menuBtn.setAttribute('aria-expanded', String(shouldOpen));
  mobileNav.hidden = !shouldOpen;
  mobileNav.classList.toggle('is-active', shouldOpen);
  if (shouldOpen) {
    const firstInteractive = mobileNav.querySelector('a, button');
    firstInteractive?.focus();
  }
}
```

(c) Interaction Implementation (Click and Hover Support)

JavaScript

```
// 1. Click Toggle Interaction
menuBtn.addEventListener('click', (e) => {
  e.stopPropagation();
  toggleNavigation();
});
// 2. Keyboard Support (Escape to Close)
document.addEventListener('keydown', (event) => {
  if (event.key === 'Escape' && menuBtn.getAttribute('aria-expanded') === 'true') {
    toggleNavigation(false);
    menuBtn.focus();
  }
});
// 3. Click-Outside to Dismiss
document.addEventListener('click', (event) => {
  const isMenuOpen = menuBtn.getAttribute('aria-expanded') === 'true';
  if (isMenuOpen && !mobileNav.contains(event.target) && event.target !==
menuBtn) {
    toggleNavigation(false);
  }
});
// 4. Desktop Dropdown Pointer Hover Support
```

```
const dropdownContainers = document.querySelectorAll('.has-dropdown');
dropdownContainers.forEach(container => {
  container.addEventListener('pointerenter', () => {
    container.classList.add('dropdown-visible');
  });
  container.addEventListener('pointerleave', () => {
    container.classList.remove('dropdown-visible');
  });
});
```

(d) Performance Evaluation and Optimization

- **Event Delegation:** Rather than attaching event listeners to each link inside the drawer menu, bind a single listener to the parent container (mobileNav.addEventListener('click', ...)). This conserves browser memory and handles dynamically added sub-links cleanly.
- **Debouncing & Passive Listeners:** For resize or scroll interactions related to menus, use passive: true flags (window.addEventListener('scroll', handler, { passive: true })) to prevent scroll blocking on the UI thread.
- **Separation of Concerns:** Keep animation logic in CSS via class toggling (is-active). Avoid manipulating layout geometry (element.style.height) directly in JavaScript execution loops.

